Glossary

abstract
Something that cannot be directly instantiated; the opposite of concrete.

abstraction
(1) The creation of a view or model that suppresses unnecessary details to focus on a specific set of details of interest. (2) The essential characteristics of an entity that distinguish it from all other kinds of entities. An abstraction defines a boundary relative to the perspective of the viewer.

acceptance
An action by which the customer accepts ownership of software products as a partial or complete performance of a contract.

action
The specification of an executable statement that forms an abstraction of a computational procedure. An action typically results in a change in the state of the system.

action sequence
An expression that resolves to a sequence of actions.

action state
A state that represents the execution of an atomic action, typically the invocation of an operation.

activation
The execution of an action.

activity
A unit of work a role may be asked to perform.

activity graph
A special case of a state machine that is used to model processes involving one or more classifiers. Contrast: statechart diagram. Synonym: activity diagram.

actor (instance)
Someone or something outside the system or business that interacts with the system or business.

actor class
Defines a set of actor instances, in which each actor instance plays the same role in relation to the system or business. A coherent set of roles that users of use cases play when interacting with these use cases. An actor has one role for each use case with which it communicates.

actor-generalization
An actor-generalization from an actor class (descendant) to another actor class (ancestor) indicates that the descendant inherits the role the ancestor can play in a use case.

aggregation
An association that models a whole-part relationship between an aggregate (the whole) and its parts.

analysis
The part of the software development process whose primary purpose is to formulate a model of the problem domain. Analysis focuses on what to do; design focuses on how to do it. See design.

analysis class
An abstraction of a role played by a design element in the system, typically within the context of a use-case realization. Analysis classes may provide an abstraction for several roles, representing the common behavior of those roles. Analysis classes typically evolve into one or more design elements (e.g., design classes or capsules, or design subsystems).

analysis & design
A core workflow in the Rational Unified Process, whose purpose is to show how the system's use cases will be realized in implementation; (general) activities during which strategic and tactical decisions are made to meet the required functional and quality requirements of a system. For the result of analysis and design activities, see "Design Model."

architectural baseline
The baseline at the end of the Elaboration phase, at which time the foundation structure and behavior of the system is stabilized.

architectural view
A view of the system architecture from a given perspective; focuses primarily on structure, modularity, essential components, and the main control flows.

architecture
(1)The highest-level concept of a system in its environment [IEEE]. The architecture of a software system (at a given point in time) is its organization or structure of significant components interacting through interfaces, those components being composed of successively smaller components and interfaces.
(2) The organizational structure of a system. Architecture can be recursively decomposed into parts that interact through interfaces, relationships that connect parts, and constraints for assembling parts. Parts that interact through interfaces include classes, components, and subsystems.

artifact
A piece of information that (1) is produced, modified, or used by a process; (2) defines an area of responsibility; and (3) is subject to version control. An artifact can be a model, a model element, or a document. A document can enclose other documents. A piece of information that is used or produced by a software development process. An artifact can be a model, a description, or software. Synonym: product.

artifact guidelines
A description of how to work with a particular artifact, including how to create and revise the artifact.

artifact set
A set of related artifacts which presents one aspects of the system. Artifact sets cut across core workflows, as several artifacts are used in a number of core workflows (e.g., the Risk List, the Software Architecture Document, and the Iteration Plan).

association
A relationship that models a bi-directional semantic connection among instances. The semantic relationship between two or more classifiers that specifies connections among their instances.

association class
A model element that has both association and class properties. An association class can be seen as an association that also has class properties or as a class that also has association properties.

association end
The endpoint of an association, which connects the association to a classifier.

attribute
(1) An attribute defined by a class represents a named property of the class or its objects. An attribute has a type that defines the type of its instances.
(2) A feature within a classifier that describes a range of values that instances of the classifier may hold.

attribute value
Information assigned to a requirement attribute. For example, the attribute *priority* can be assigned the values Low, Medium and High. Also referred to as r*equirement attribute value*.

baseline
A reviewed and approved release of artifacts that constitutes an agreed basis for further evolution or development and that can be changed only through a formal procedure, such as change management and configuration control.

behavior
The observable effects of an operation or event, including its results.

behavioral feature
A dynamic feature of a model element, such as an operation or method.

behavioral model aspect
A model aspect that emphasizes the behavior of the instances in a system, including their methods, collaborations, and state histories.

boundary class
A class used to model communication between the system's environments and its inner workings.

business actor (instance)
Someone or something outside the business that interacts with the business.

business actor class
Defines a set of business-actor instances, in which each business-actor instance plays the same role in relation to the business.

business creation
To perform business engineering where the goal is to create a new business process, a new line of business or a new organization.

business engineering
A set of techniques a company uses to design its business according to specific goals. Business engineering techniques can be used for both business reengineering, business improvement, and business creation.

business entity
A business entity represents a "thing" handled or used by business workers.

business improvement
To perform business engineering where the work of change is local and does not span the entire business. It involves trimming costs and lead times and monitoring service and quality.

business modeling
Encompasses all modeling techniques you can use to visually model a business. These are a subset of the techniques you may use to perform business engineering.

business object model
An object model describing the realization of business use cases.

business process
A group of logically related activities that use the resources of the organization to provide defined results in support of the organization's objectives. In the Rational Unified Process, we define business processes using business use cases, which show the expected behavior of the business, and business use-case realizations, which show how that behavior is realized by business workers and business entities. See also process.

business process engineering
See business engineering.

business reengineering
To perform business engineering where the work of change includes taking a comprehensive view of the entire existing business and thinking through why you do what you do. You question all existing business processes and try to find completely new ways of reconstructing them to achieve radical improvements. Other names for this are business process reengineering (BPR) and process innovation.

business rule
A declaration of policy or condition that must be satisfied within the business.

business use case (class)
A business use case defines a set of business use-case instances, where each instance is a sequence of actions a business performs that yields an observable result of value to a particular business actor. A business use-case class contains all main, alternate workflows related to producing the "observable result of value."

business use-case instance
A sequence of actions performed by a business that yields an observable result of value to a particular business actor.

business use-case model
A model of the business intended functions. The business use-case model is used as an essential input to identify roles and deliverables in the organization.

business use-case package
A business use-case package is a collection of business use cases, business actors, relationships, diagrams, and other packages; it is used to structure the business use-case model by dividing it into smaller parts.

business use-case realization
A business use-case realization describes how the workflow of a particular business use case is realized within the business object model, in terms of collaborating business objects.

business worker
A business worker represents a role or set of roles in the business. A business worker interacts with other business workers and manipulates business entities while participating in business use-case realizations.

CMM
The Capability Maturity Model, a framework for effective software process developed at the Software Engineering Institute at Carnegie-Mellon. CMM for Software describes increasing levels of process maturity, from an ad-hoc, immature process to a mature, disciplined one.

change control board (CCB)
The role of the CCB is to provide a central control mechanism to ensure that every change request is properly considered, authorized, and coordinated.

change management
The activity of controlling and tracking changes to artifacts. See also: scope management.

change request (CR)
A general term for any request from a stakeholder to change an artifact or process. Documented in the Change Request is information on origin and impact of the current problem, the proposed solution, and its cost. See also: enhancement request, defect.

checkpoints
A set of conditions that well-formed artifacts of a particular type should exhibit. May also be stated in the form of questions which should be answered in the affirmative.

child
In a generalization relationship, the specialization of another element, the parent. Contrast: parent.

circular traceability relationship
A relationship between a requirement and itself, or an indirect relationship that leads back to a previously traced-from node. Traceability relationships cannot have circular references.

class
A description of a set of objects that share the same attributes operations, methods, relationships, and semantics. A class may use a set of interfaces to specify collections of operations it provides to its environment. See: interface.

class diagram
A diagram that shows a collection of declarative (static) model elements, such as classes, types, and their contents and relationships.

classifier
A mechanism that describes behavioral and structural features. Classifiers include interfaces, classes, datatypes, and components.

collaboration
Interaction among objects.

collaboration diagram
A collaboration diagram describes a pattern of interaction among objects; it shows the objects participating in the interaction by their links to each other and the messages they send to each other.

comment
An annotation attached to an element or a collection of elements. A note has no semantics. Contrast: constraint.

communicates-association
An association between an actor class and a use case class, indicating that their instances interact. The direction of the association indicates the initiator of the communication (Unified Process convention).

communication association
In a deployment diagram an association between nodes that implies a communication. See: deployment diagram.

component
(1) A nontrivial, nearly independent, and replaceable part of a system that fulfills a clear function in the context of a well-defined architecture. A component conforms to and provides the physical realization of a set of interfaces. (2) A physical, replaceable part of a system that packages implementation and conforms to and provides the realization of a set of interfaces. A component represents a physical piece of implementation of a system, including software code (source, binary, or executable) or equivalents such as scripts or command files.

component diagram
A diagram that shows the organizations and dependencies among components.

component-based development (CBD)
The creation and deployment of software-intensive systems assembled from components as well as the development and harvesting of such components.

concrete
An entity in a configuration that satisfies an end-use function and can be uniquely identified at a given reference point. (ISO)

configuration
(1) general: The arrangement of a system or network as defined by the nature, number, and chief characteristics of its functional units; applies to both hardware or software configuration.
(2) The requirements, design, and implementation that define a particular version of a system or system component. See configuration management.

configuration item
An entity in a configuration that satisfies an end-use function and can be uniquely identified at a given reference point. (ISO)

configuration management
A supporting process whose purpose is to identify, define, and baseline items; control modifications and releases of these items; report and record status of the items and modification requests; ensure completeness, consistency and correctness of the items; and control storage, handling and delivery of the items. (ISO)

constraint
A semantic condition or restriction. Certain constraints are predefined in the UML, others may be user defined. Constraints are one of three extensibility mechanisms in UML. See: tagged value, stereotype.

construction
The third phase of the Unified Process, in which the software is brought from an executable architectural baseline to the point at which it is ready to be transitioned to the user community.

container
(1) An instance that exists to contain other instances, and that provides operations to access or iterate over its contents. (for example, arrays, lists, sets).
(2) A component that exists to contain other components.

context
A view of a set of related modeling elements for a particular purpose, such as specifying an operation.

core workflow
One of nine core workflows in the Rational Unified Process: Business Engineering, Requirements, Analysis & Design, Implementation, Test, Deployment, Configuration & Change Management, Project Management, Environment. An abstract business use case of the Software-Engineering Business.

customer
A person or organization, internal or external to the producing organization, who takes financial responsibility for the system. In a large system this may not be the end user. The customer is the ultimate recipient of the developed product and its artifacts. See also: stakeholder.

cycle
One complete pass through the four phases: inception, elaboration, construction and transition. The span of time between the beginning of the inception phase and the end of the transition phase.

defect
An anomaly, or flaw, in a delivered work product. Examples include such things as omissions and imperfections found during early lifecycle phases and symptoms of faults contained in software sufficiently mature for test or operation. A defect can be any kind of issue you want tracked and resolved. See also: change request.

deliverable
An output from a process that has a value, material or otherwise, to a customer or other stakeholder.

dependency
A relationship between two modeling elements, in which a change to one modeling element (the independent element) will affect the other modeling element (the dependent element).

deployment
A core process workflow in the software-engineering process, whose purpose is to ensure a successful transition of the developed system to its users. Included are artifacts such as training materials and installation procedures.

deployment diagram
A diagram that shows the configuration of run-time processing nodes and the components, processes, and objects that live on them. Components represent run-time manifestations of code units. See: component diagram.

deployment view
An architectural view that describes one or several system configurations; the mapping of software components (tasks, modules) to the computing nodes in these configurations.

design
The part of the software development process whose primary purpose is to decide how the system will be implemented. During design, strategic and tactical decisions are made to meet the required functional and quality requirements of a system. See analysis.

design model
An object model describing the realization of use cases; serves as an abstraction of the implementation model and its source code.

design package
A collection of classes, relationships, use-case realizations, diagrams, and other packages; it is used to structure the design model by dividing it into smaller parts.

design subsystem
A model element that has the semantics of a package (it can contain other model elements) and a class (it has behavior). The behavior of the subsystem is provided by classes or other subsystems it contains. A subsystem realizes one or more interfaces, which define the behavior it can perform. Contrast: design package.

developer
A person responsible for developing the required functionality in accordance with project-adopted standards and procedures. This can include performing activities in any of the requirements, analysis and design, implementation, and test workflows.

development case
The software-engineering process used by the performing organization. It is developed as a configuration, or customization, of the Unified Process product, and adapted to the project's needs.

development process
A set of partially ordered steps performed for a given purpose during software development, such as constructing models or implementing models.

diagram
A graphical depiction of all or part of a model. A graphical presentation of a collection of model elements, most often rendered as a connected graph of arcs (relationships) and vertices (other model elements). UML supports the following diagrams: class diagram, object diagram, use-case diagram, sequence diagram, collaboration diagram, statechart diagram, activity diagram, component diagram, and deployment diagram.

document
A document is a collection of information that is intended to be represented on paper, or in a medium using a paper metaphor. The paper metaphor includes the concept of pages, and it has either an implicit or explicit sequence of contents. The information is in text or two-dimensional pictures. Examples of paper metaphors are word processor documents, spreadsheets, schedules, Gantt charts, Web pages, or overhead slide presentations.

document description
Describes the contents of a particular document.

document template
A concrete tool template, such as a Adobe® FrameMaker™ or Microsoft® Word™ template.

domain
An area of knowledge or activity characterized by a family of related systems. An area of knowledge or activity characterized by a set of concepts and terminology understood by practitioners in that area.

domain model
A domain model captures the most important types of objects in the context of the domain. The domain objects represent the entities that exist or events that transpire in the environment in which the system works. The domain model is a subset of the business object model.

elaboration
The second phase of the process where the product vision and its architecture are defined.

e-Business
Either (1) the transaction of business over an electronic medium such as the Internet or (2) a business that uses Internet technologies and network computing in their internal business processes (via intranets), their business relationships (via extranets), and the buying and selling of goods, services, and information (via electronic commerce).

element
An atomic constituent of a model.

enclosed document
A document can be enclosed by another document to collect a set of documents into a whole; the enclosing document as well as the individual enclosures are regarded as separate artifacts.

enhancement request
A type of stakeholder request that specifies a new feature or functionality of the system. See also: change request

enumeration
A list of named values used as the range of a particular attribute type. For example, RGBColor = {red, green, blue}. Boolean is a predefined enumeration with values from the set {false, true}.

event
The specification of a significant occurrence that has a location in time and space. In the context of state diagrams, an event is an occurrence that can trigger a transition.

environment
A core supporting workflow in the software-engineering process, whose purpose is to define and manage the environment in which the system is being developed. Includes process descriptions, configuration management, and development tools.

evolution
The life of the software after its initial development cycle; any subsequent cycle, during which the product evolves.

evolutionary
An iterative development strategy that acknowledges that user needs are not fully understood and therefore requirements are refined in each succeeding iteration (elaboration phase).

executable architecture
An executable architecture is a partial implementation of the system, built to demonstrate selected system functions and properties, in particular those satisfying nonfunctional requirements. It is built during the elaboration phase to mitigate risks related to performance, throughput, capacity, reliability and other 'ilities', so that the complete functional capability of the system may be added in the construction phase on a solid foundation, without fear of breakage. It is the intention of the Rational Unified Process that the executable architecture be built as an evolutionary prototype, with the intention of retaining what is found to work (and satisfies requirements), and making it part of the deliverable system.

extend
A relationship from an extension use case to a base use case, specifying how the behavior defined for the extension use case can be inserted into the behavior defined for the base use case.

extend-relationship
An extend-relationship from a use-case class A to a use-case class B indicates that an instance of B may include (subject to specific conditions specified in the extension) the behavior specified by A. Behavior specified by several extenders of a single target use case can occur within a single use-case instance.

fault
An accidental condition that causes a component in the implementation model to fail to perform its required behavior. A fault is the root cause of one or more defects.

feature
An externally observable service provided by the system that directly fulfills a stakeholder need. A property, like operation or attribute, which is encapsulated within a classifier, such as an interface, a class, or a datatype.

framework
A micro-architecture that provides an extensible template for applications within a specific domain.

FURPS
An acronym representing categories for assessing product quality: Functionality, Usability, Reliability, Performance, Supportability.

generalization
A taxonomic relationship between a more general element and a more specific element. The more specific element is fully consistent with the more general element and contains additional information. An instance of the more specific element may be used where the more general element is allowed. See: inheritance.

generation
Final release at the end of a cycle.

graphical user interface (GUI)
A type of interface that enables users to communicate with a program by manipulating graphical features, rather than by entering commands. Typically, a GUI includes a combination of graphics, pointing devices, menu bars and other menus, overlapping windows, and icons.

green-field development
Development "starting from scratch", as opposed to "evolution of an existing system" or "reengineering of a legacy piece". (Originated from the transformation that takes place when building a new factory on an undeveloped site - with grass on it.).

IEEE ("I triple E")
Institute of Electronics and Electrical Engineers, the hardware and software standards body, sets standards for such measurable items as acceptability criteria, performance benchmarks, and configuration management.

ISO 9000: ("eye-so")
International Organization for Standardization's set of industry specific standards. The ISO works to establish global standards for communications and information exchange. The ISO 9000 standards are a set of documents that can be used for quality assurance purposes.

implementation
(1) A core process workflow in the software-engineering process, whose purpose is to implement and unit test the classes. (2) A definition of how something is constructed or computed. For example, a class is an implementation of a type; a method is an implementation of an operation.

implementation model
The implementation model is a collection of components, and the implementation subsystems that contain them.

implementation subsystem
A collection of components and other implementation subsystems. It is used to structure the implementation model by dividing it into smaller parts.

implementation view
An architectural view that describes the organization of the static software elements (code, data, and other accompanying artifacts) on the development environment, in terms of packaging, layering, and configuration management (ownership, release strategy, and so on). In the Unified Process it is a view on the implementation model.

inception
The first phase of the Unified Process, in which the seed idea, request for proposal, for the previous generation is brought to the point of being (at least internally) funded to enter the elaboration phase.

include
A relationship from a base use case to an inclusion use case, specifying how the behavior defined for the inclusion use case can be inserted into the behavior defined for the base use case.

include-relationship
An include-relationship is a relationship from a base use case to an inclusion use case, specifying how the behavior defined for the inclusion use case is explicitly inserted into the behavior defined for the base use case.

increment
The difference (delta) between two releases at the end of subsequent iterations.

incremental
Qualifies an iterative development strategy in which the system is built by adding more and more functionality at each iteration.

inheritance
(1) The mechanism that makes generalization possible; a mechanism for creating full class descriptions out of individual class segments. (2) The mechanism by which more specific elements incorporate structure and behavior of more general elements related by behavior. See generalization.

input
An artifact used by a process. See static artifact.

inspection
A formal evaluation technique in which some artifact (model, document, software) is examined by a person or group other than the originator, to detect faults, violations of development standards, and other problems.

instance
An individual entity satisfying the description of a class or type. An entity to which a set of operations can be applied and which has a state that stores the effects of the operations. See: object.

integration
The software development activity in which separate software components are combined into an executable whole.

integration build plan
Defines the order in which components are to be implemented and integrated in a specific iteration. Enclosed in the Iteration Plan.

interaction
A specification of how stimuli are sent between instances to perform a specific task. The interaction is defined in the context of collaboration. See collaboration.

interaction diagram
A generic term that applies to several types of diagrams that emphasize object interactions. These include: collaboration diagrams, sequence diagrams, and activity diagrams.

interface
A collection of operations that are used to specify a service of a class or a component. A named set of operations that characterize the behavior of an element.

iteration
A distinct sequence of activities with a base lined plan and valuation criteria resulting in a release (internal or external).

layer
A specific way of grouping packages in a model at the same level of abstraction.

link
A semantic connection among a tuple of objects. An instance of an association. See: association.

link end
An instance of an association end. See: association end.

logical view
An architectural view that describes the main classes in the design of the system: major business-related classes, and the classes that define key behavioral and structural mechanisms (persistency, communications, fault-tolerance, user-interface). In the Unified Process, the logical view is a view of the design model.

management
A core supporting workflow in the software-engineering process, whose purpose is to plan and manage the development project.

metamodel
A model that defines the language for expressing a model.

method
(1) A regular and systematic way of accomplishing something; the detailed, logically ordered plans or procedures followed to accomplish a task or attain a goal.
(2) UML 1.1: The implementation of an operation, the algorithm or procedure that effects the results of an operation. The implementation of an operation. It specifies the algorithm or procedure associated with an operation.

milestone
The point at which an iteration formally ends; corresponds to a release point.

model
A semantically closed abstraction of a system. In the Unified Process, a complete description of a system from a particular perspective ('complete' meaning you don't need any additional information to understand the system from that perspective); a set of model elements. Two models cannot overlap.

model aspect
A dimension of modeling that emphasizes particular qualities of the metamodel. For example, the structural model aspect emphasizes the structural qualities of the metamodel.

model element
An element that is an abstraction drawn from the system being modeled.

modeling conventions
How concepts will be represented, restrictions on the modeling language that the project team management has decided upon (i.e. dictums such as "Do not use inheritance between subsystems."; "Do not use extend or include associations in the Use Case Model."; "Do not use the friend construct in C++."). Presented in the Software Architecture Document.

name
A string used to identify a model element.

namespace
A part of the model in which the names may be defined and used. Within a namespace, each name has a unique meaning. See: name.

object
An entity with a well-defined boundary and identity that encapsulates state and behavior. State is represented by attributes and relationships; behavior is represented by operations, methods, and state machines. An object is an instance of a class.

object diagram
A diagram that encompasses objects and their relationships at a point in time. An object diagram may be considered a special case of a class diagram or a collaboration diagram. See: class diagram, collaboration diagram.

object model
An abstraction of a system's implementation.

organization unit
A collection of business workers, business entities, relationships, business use-case realizations, diagrams, and other organization units. It is used to structure the business object model by dividing it into smaller parts.

originator
An originator is anyone who submits a change request (CR). The standard change request mechanism requires the originator to provide information on the current problem, and a proposed solution in accordance with the change request form.

output
Any artifact that is the result of a process step. See deliverable.

package
A general-purpose mechanism for organizing elements into groups. Packages may be nested within other packages.

parent
In a generalization relationship, the generalization of another element, the child. Contrast: child.

participates
The connection of a model element to a relationship or to a reified relationship. For example, a class participates in an association, an actor participates in a use case.

pattern
(1)In software, a solution template (for structure, behavior) that has proven useful in at least one practical context, or in several contexts, to be worthy of being called a pattern. (2) In UML, a parameterized collaboration is used to represent a pattern.

phase
The time between two major project milestones, during which a well-defined set of objectives is met, artifacts are completed, and decisions are made to move or not move into the next phase.

postcondition
(1) A textual description defining a constraint on the system when a use case has terminated.
(2) A constraint that must be true at the completion of an operation.

precondition
(1) A textual description defining a constraint on the system when a use case may start.
(2) A constraint that must be true when an operation is invoked.

process
(1) A set of partially ordered steps intended to reach a goal; in software engineering the goal is to build a software product or to enhance an existing one; in process engineering, the goal is to develop or enhance a process model; corresponds to a business use case in business engineering.
(2) A software development process—the steps and guidelines by which to develop a system.
(3) To execute an algorithm or otherwise handle something dynamically.

process view
An architectural view that describes the concurrent aspect of the system: tasks (processes) and their interactions.

product
Software that is the result of development, and some of the associated artifacts (documentation, release medium, training).

product champion
A high-ranking individual who owns the vision of the product and acts as an advocate between development and the customer.

product requirements document (PRD)
A high level description of the product (system), its intended use, and the set of features it provides.

project
Projects are performed by people, constrained by limited resources, and planned, executed, and controlled. A project is a temporary endeavor undertaken to create a unique product or service. Temporary means that every project has a definite beginning and a definite ending. Unique means that the product or service is different in some distinguishing way from all similar products and services. Projects are often critical components of the performing organizations' business strategy.

project manager
The worker with overall responsibility for the project. The Project Manager needs to ensure tasks are scheduled, allocated and completed in accordance with project schedules, budgets and quality requirements.

Project Review Authority (PRA)
The organizational entity to which the Project Manager reports. The PRA is responsible for ensuring that a software project complies with policies, practices and standards.

property
A named value denoting a characteristic of an element. A property has semantic impact. Certain properties are predefined in the UML; others may be user defined. See: tagged value.

prototype
A release that is not necessarily subject to change management and configuration control.

quality assurance (QA)
The function of Quality Assurance is the responsibility of (reports to) the Project Manager and is responsible for ensuring that project standards are correctly and verifiably followed by all project staff.

query
A method for filtering and sorting requirements by limiting the values of one or more attributes or by limiting traceability, and specifying the order in which the filtered requirements display. For example, you might create a query such that only requirements having a high or medium customer priority are displayed.

rank
An attribute of a use case or scenario that describes its impact on the architecture, or its importance for a release.

rationale
A statement, or explanation of the reasons for a choice.

refinement
A relationship that represents a fuller specification of something that has already been specified at a certain level of detail. For example, a design class is a refinement of an analysis class.

relationship
A semantic connection among model elements. Examples of relationships include associations and generalizations.

release
A subset of the end product that is the object of evaluation at a major milestone. A release is a stable, executable version of product, together with any artifacts necessary to use this release, such as release notes or installation instructions. A release can be internal or external. An internal release is used only by the development organization, as part of a milestone, or for a demonstration to users or customers. An external release (or delivery) is delivered to users. A release is not necessarily a complete product, but can just be one step along the way, with its usefulness measured only from an engineering perspective. Releases act as a forcing function that drives the development team to get closure at regular intervals, avoiding the "90 percent done, 90 percent remaining" syndrome. See also: prototype, baseline.

release manager
A release manager is responsible for ensuring that all software assets are controlled and configurable into internal and external releases as required.

report
An automatically generated description, describing one or several artifacts. A report is not an artifact in itself. A report is in most cases a transitory product of the development process and a vehicle to communicate certain aspects of the evolving system; it is a snapshot description of artifacts that are not documents themselves.

repository
A storage place for object models, interfaces, and implementations.

requirement
A requirement describes a condition or capability to which a system must conform, either derived directly from user needs or stated in a contract, standard, specification, or other formally imposed document. A desired feature, property, or behavior of a system.

requirement attribute
Information associated with a particular requirement providing a link between the requirement and other project elements, e.g., priorities, schedules, status, design elements, resources, costs, hazards.

requirement type
A categorization of requirements—for example, stakeholder need, feature, use case, supplementary requirement, test requirement, documentation requirement, hardware requirement, software requirement, and so on—based on common characteristics and attributes.

requirements
A core process workflow in the software-engineering process, whose purpose is to define what the system should do. The most significant activities are to develop a vision, a use-case model and software requirements specifications.

requirements management
A systematic approach to eliciting, organizing, and documenting the requirements of the system, and establishing and maintaining agreement between the customer and the project team on the changing requirements of the system.

requirements tracing
The linking of a requirement to other requirements and to other associated project elements.

responsibility
A contract or obligation of a classifier.

result
Synonym of output. See also deliverable.

reuse
(1) Further use or repeated use of an artifact. 2) The use of a pre-existing artifact.

review
A review is a group activity carried out to discover potential defects and to assess the quality of a set of artifacts.

risk
An ongoing or upcoming concern that has a significant probability of adversely affecting the success of major milestones.

role
(1) A definition of the behavior and responsibilities of an individual, or a set of individuals working together as a team, within the context of a software engineering organization.
(2) The named specific behavior of an entity participating in a particular context. A role may be static (for example, an association end) or dynamic (for example, a collaboration role).

scenario
(1) A described use-case instance, a subset of a use case. (2) A specific sequence of actions that illustrates behaviors. A scenario may be used to illustrate an interaction or the execution of a use case instance. See: interaction.

scope management
The process of prioritizing and determining the set of requirements that can be implemented in a particular release cycle based on the resources and time available. This process continues throughout the lifecycle of the project as changes occur. See also: change management.

sequence diagram
A diagram that shows object interactions arranged in time sequence. In particular, it shows the objects participating in the interaction and the sequence of messages exchanged. Unlike a collaboration diagram, a sequence diagram includes time sequences but does not include object relationships.

software architecture
Software architecture encompasses the significant decisions about the organization of a software system, the selection of the structural elements and their interfaces by which the system is composed together with their behavior as specified in the collaboration among those elements, the composition of the structural and behavioral elements into progressively larger subsystems, the architectural style that guides this organization, these elements and their interfaces, their collaborations, and their composition. Software architecture is not only concerned with structure and behavior, but also with usage, functionality, performance, resilience, reuse, comprehensibility, economic and technology constraints and tradeoffs, and aesthetic concerns.

Software Engineering Process Authority (SEPA)
The organizational entity with responsibility for process definition, assessment and improvement.

software requirement
A specification of an externally observable behavior of the system; for example, inputs to the system, outputs from the system, functions of the system, attributes of the system, or attributes of the system environment.

software requirements specifications (SRS)
A set of requirements that completely defines the external behavior of the system to be built—sometimes called a functional specification.

software specification review (SSR)
In the waterfall lifecycle, the major review held when the software requirements specification is complete.

specification
A declarative description of what something is or does. Contrast: implementation.

stakeholder
An individual who is materially affected by the outcome of the system.

stakeholder need
A business or operational problem (opportunity) that must be fulfilled to justify purchase or use.

stakeholder request
A request of any type from a stakeholder. For example, Change Request, enhancement request, request for a requirement change, defect.

state
A condition or situation during the life of an object during which it satisfies some condition, performs some activity, or waits for some event.

statechart diagram
A diagram that shows a state machine. See: state machine.

state diagram
A diagram that shows a state machine. See: state machine.

state machine
A state machine specifies the behavior of a model element, defining its response to events and the life cycle of the object. A behavior that specifies the sequences of states that an object or an interaction goes through during its life in response to events, together with its responses and actions.

static artifact
An artifact that is used, but not changed, by a process.

stereotype
A meta-classification of an element. A new type of modeling element that extends the semantics of the metamodel. Stereotypes must be based on existing types or classes in the metamodel. Stereotypes may extend the semantics, but not the structure of pre-existing types and classes. Certain stereotypes are predefined in the UML; others may be user defined.

structural feature
A static feature of a model element, such as an attribute.

structural model aspect
A model aspect that emphasizes the structure of the objects in a system, including their types, classes, relationships, attributes, and operations.

stub
A component containing functionality for testing purposes. A stub is either a pure "dummy", just returning some predefined values, or it is "simulating" a more complex behavior.

subsystem
A model element that has the semantics of a package, such that it can contain other model elements, and a class, such that it has behavior.

suspect relationship state
A state applied to a traceability or hierarchical relationship when a change occurs to one or both of the requirements in the relationship. A suspect relationship state indicates that, because of the modification to the requirement(s), the other requirement in the relationship may need modification as well.

system
As an instance, an executable configuration of a software application or software application family; the execution is done on a hardware platform. As a class, a particular software application or software application family that can be configured and installed on a hardware platform. In a general sense, an arbitrary system instance.

system analyst
An individual who leads and coordinates requirements elicitation and use-case modeling by outlining the system's functionality and delimiting the system.

tagged value
The explicit definition of a property as a name-value pair. In a tagged value, the name is referred as the tag. Certain tags are predefined in the UML; others may be user defined. Tagged values are one of three extensibility mechanisms in UML. See: constraint, stereotype.

team leader
The team leader is the interface between project management and developers. The team leader is responsible for ensuring that a task is allocated and monitored to completion. The team leader is responsible for ensuring that development staff follow project standards and adhere to project schedules.

technical authority
The project's technical authority has the authority and technical expertise to arbitrate on if, and how, a change request is to be implemented. The technical authority defines change tasks and estimates the effort of engineering the work tasks (corresponding to a change request).

template
A predefined structure for an artifact. Synonym: parameterized element.

test
A core process workflow in the software-engineering process whose purpose is to integrate and test the system.

test case
A set of test inputs, execution conditions, and expected results developed for a particular objective, such as to exercise a particular program path or to verify compliance with a specific requirement.

test coverage
The degree to which a given test or set of tests addresses all specified test cases for a given system or component.

test driver
A software module or application used to invoke a test item and, often, provide test inputs (data), control and monitor execution, and report test results. A test driver automates the execution of test procedures.

test procedure
A test procedure is a set of detailed instructions for the setup, execution, and evaluation of results for a given test case.

time
A value representing an absolute or relative moment in time.

time event
An event that denotes the time elapsed since the current state was entered. See: event.

time expression
An expression that resolves to an absolute or relative value of time.

tollgate
A major milestone in which decisions are made about funding for a project.

tool mentor
A description that provides practical guidance on how to perform specific process activities or steps using a specific software tool.

trace
A dependency that indicates a historical or process relationship between two elements that represent the same concept without specific rules for deriving one from the other.

traceability
The ability to trace a project element to other related project elements, especially those related to requirements. Project elements involved in traceability are called traceability items.

traceability item
Any project element that needs to be explicitly traced from another project element in order to keep track of the dependencies between them. With respect to Rational RequisitePro this definition can be rephrased as: any project element represented within RequisitePro by an instance of a RequisitePro requirement type.

traceability matrix
A matrix that illustrates the relationships between two types of requirements.

traceability tree
A hierarchical display of all requirements traced to or from a requirement (depending on the direction of the tree). Also referred to as a *Tree view* or *Tree Report*.

transaction
A unit of processing consisting of one or more application programs initiated by a single request. A transaction can require the initiation of one or more tasks for its execution.

transaction processing
A style of computing that supports interactive applications in which requests submitted by users are processed as soon as they are received. Results are returned to the requester in a relatively short period of time. A transaction processing system supervises the sharing of resources for processing multiple transactions at the same time.

transition
The fourth phase of the process in which the software is turned over to the user community.

type
Description of a set of entities that share common characteristics, relations, attributes, and semantics.

Unified Modeling Language (UML)
Unified Modeling Language [UML99].

uniform resource locator (URL)
A standard identifier for a resource on the World Wide Web, used by Web browsers to initiate a connection. The URL includes the communications protocol to use, the name of the server, and path information identifying the objects to be retrieved on the server.

usage
A dependency in which one element (the client) requires the presence of another element (the supplier) for its correct functioning or implementation.

use case (class)
(1) A use case defines a set of use-case instances, where each instance is a sequence of actions a system performs that yields an observable result of value to a particular actor. A use-case class contains all main, alternate flows of events related to producing the "observable result of value". Technically, a use-case is a class whose instances are scenarios.
(2) (UML) The specification of a sequence of actions, including variants, that a system (or other entity) can perform, interacting with actors of the system. See use-case instances.

use-case diagram
A diagram that shows the relationships among actors and use cases within a system.

use-case instance
(1) A sequence of actions performed by a system that yields an observable result of value to a particular actor. (2) The performance of a sequence of actions being specified in a use case. An instance of a use case. See: use-case class.

use-case model
A model that describes a system's functional requirements in terms of use cases.

use-case package
A use-case package is a collection of use cases, actors, relationships, diagrams, and other packages; it is used to structure the use-case model by dividing it into smaller parts.

use-case realization
A use-case realization describes how a particular use case is realized within the design model, in terms of collaborating objects.

use-case view
An architectural view that describes how critical use cases are performed in the system, focusing mostly on architecturally significant components (objects, tasks, nodes). In the Unified Process, it is a view of the use-case model.

user
An individual or entity that directly uses the system or the output of a system to accomplish an observable result.

user interface (UI)
(1) The hardware, software, or both, that enables a user to interact with a computer. (2) The term user interface typically refers to the visual presentation and its underlying software with which a user interacts.

version
A variant of some artifact; later versions of an artifact typically expand on earlier versions.

view
(1) A simplified description (an abstraction) of a model, which is seen from a given perspective or vantage point and omits entities that are not relevant to this perspective. See also architectural view. (2) (UML) A projection of a model, which is seen from a given perspective or vantage point and omits entities that are not relevant to this perspective.

vision
The user's or customer's view of the product to be developed, specified at the level of key stakeholder needs and features of the system.

Vision document
A high level description of the *product* (*system*), the key user needs it addresses, its intended use, and the set of *features* it provides.

Web application
A system that uses the Internet as the primary means of communication between the system users and the system. See also Web system.

Web browser
A piece of software that runs on a client that allows a user to request and render HTML pages.

Web server
The server component of the World Wide Web. It is responsible for servicing requests for information from Web browsers. The information can be a file retrieved from the server's local disk or generated by a program called by the server to perform a specific application function.

Web site
A Web system that is all on one server. Users navigate the Web site with a browser.

Web system
A hyper media system that contains pages of information that are linked to each other in the form of a graph, as opposed to being hierarchical or linear. A web system can manifest itself as a Web server that can be accessed through a browser.

work breakdown structure
The planning framework; a project decomposition into units of work from which cost, artifacts, and activities can be allocated and tracked.

work guideline
A description that provides practical guidance on how to perform an activity or set of activities. It usually considers techniques that are useful during the activity.

worker
A definition of the behavior and responsibilities of an individual, or a set of individuals working together as a team, within the context of a software engineering organization. The worker represents a role played by individuals on a project and defines how they carry out work.

workflow

The sequence of activities performed in a business that produces a result of observable value to an individual actor of the business.

workflow detail

A grouping of activities that are performed in close collaboration to accomplish some result. The activities are typically performed either in parallel or iteratively, with the output from one activity serving as the input to another activity. Workflow details are used to group activities to provide a higher level of abstraction and to improve the comprehensibility of workflows.

World Wide Web (WWW or Web)

A graphic hypertextual multimedia Internet service.